Wadie Abboud - ucu8tt
Henry Morris - yfm3bn
Ashwin Gopinathan - kaz9hp

CS 4501: Final Project - Executive Summary

Introduction

This final project is focused on the *Rabin-Karp String Matching* algorithm. The problem this algorithm addresses is that of identifying where a particular pattern exists within a string. More specifically, given a string of text and a target pattern sequence, this algorithm efficiently identifies the index of the particular pattern, if it exists.

Problem Statement

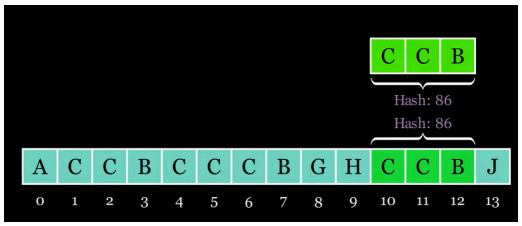
While many standard and naive solutions to this problem exist, the Rabin-Karp algorithm focuses on speed of operations, which decrease the average runtime, improving its performance when executed at scale. Most other naive, brute-force solutions work by effectively iterating through the given string and checking each possible window for the pattern. These brute-force solutions yield a time complexity of $O(nm)$, with n being the length of the string, and m being the length of the pattern. Despite skipping substrings in which the pattern is broken, this approach still falls to various cases which contain similar patterns, forcing it to iterate through the pattern multiple times unnecessarily. The key bottleneck in other approaches is the cost of each comparison, which causes unnecessarily high runtimes. Rabin-Karp seeks to address this inefficiency and optimize to an average runtime of $O(n+m)$. It does so by utilizing a rolling hash, the details of which will be described in the proceeding section.

String/pattern matching algorithms have various real world applications and uses. For instance, in bioinformatics, researchers rely on efficient string matching to scan massive genomes, often containing billions of base pairs, to pinpoint the exact locations of specific DNA mutations. Additionally, plagiarism detectors must be able to efficiently scan large blocks of text

to check for potential matches of thousands of patterns. In cybersecurity, systems must continuously parse gigabytes of server/traffic logs in real-time in order to detect known malware signatures. The key with these examples is that optimization is crucial for the algorithm to effectively achieve its purpose. By leveraging hashing and its constant comparisons, the Rabin-Karp algorithm is able to bypass the bottlenecks of the naive solutions, maintaining the runtime required to operate at scale.

Overall Approach

Similar to naive approaches, the Rabin-Karp algorithm uses a sliding window to iterate through the main string, checking substrings against a given pattern. The key differentiator, however, is the integration of a rolling hash. By converting strings into numerical hash values, the algorithm can execute comparisons in constant O(1) time.



Before the search begins, the algorithm converts the target pattern and the first m characters of the main string into integer hash values. Once these initial values are precalculated, the algorithm continuously moves the comparison window forward by one index. Instead of recalculating the hash from scratch for every new window, Rabin-Karp dynamically updates it. It does so by mathematically dropping the outgoing character's value, shifting the remaining characters, and adding the incoming character's value. Because these updates and comparisons are just integer arithmetic, they execute in constant time.

While hash functions are extremely effective at quickly verifying when two substrings are not equal, a matching hash does not completely guarantee the substrings are identical due to the rare chance of a hash collision. To account for these edge cases, Rabin-Karp treats the hash match as a filter, executing a manual, character-by-character comparison if and only if the hash

values match. Together, these optimizations allow Rabin-Karp to run in an average time complexity of O(n + m), with the O(m) stemming from the precalculation of the pattern's hash, and the O(n) resulting from the linear, constant-time iteration through the string.

Implementation

To implement the Rabin-Karp algorithm, the high-level concepts of the sliding window and rolling hash must be translated into concrete arithmetic operations. The implementation requires a few parameters: the text string T of length n, the target pattern P of length m, the alphabet size d (typically 256 to represent the extended ASCII character set), and a prime number q to use as a modulo. Applying the modulo at each arithmetic step is critical, as it prevents the hash values from exceeding the maximum integer limit of the programming language. This intermediate application of the modulo is allowed since the modulo operation has the associative property. The algorithm is executed in two distinct phases: Preprocessing and the Rolling Hash Loop.

<u>Preprocessing</u>

Before scanning the text, the algorithm must precalculate three values: the place-value multiplier for the leftmost character (h), the target hash of the pattern (H_P), and the initial hash of the first text window (H_T).

The multiplier h is mathematically defined as $h = d^{m-1} \ (mod\ q)$ and is used later to drop the outgoing character from the rolling hash. The initial hashes are calculated using Horner's method, which evaluates the polynomial hash iteratively to keep the numbers manageable.

```
// Phase 1 Pseudocode
h = 1
```

```
for i from 0 to m - 2:
        h = (h * d) % q
Hp = 0
Ht = 0
for i from 0 to m - 1:
        Hp = (d * Hp + P[i]) % q
        Ht = (d * Ht + T[i]) % q
// Pseudocode end
```

<u>Rolling Hash Loop</u>

Once the initial hashes are established, the algorithm iterates through the text string from index *0* to *n - m*. At each window position, the algorithm checks if H_P equals H_T. If the hashes match, a nested loop performs a strict, character-by-character comparison to verify the match and rule out a hash collision.

If the hashes do not match, or if a hash collision is confirmed, the window slides forward by one index. The text hash is updated in *O(1)* time using the rolling hash mathematical formula:

$$H_{new} = (d \cdot (H_{old} - c_{out} \cdot h) + c_{in}) \ (mod\ q)$$

Depending on the programming language used (such as Java or C++), subtracting the outgoing character's value can sometimes result in a negative integer before the modulo is applied. To ensure the hash remains a positive integer so that it can be indexed, *q* is simply added to any negative hash value.

```
// Phase 2 Pseudocode
for i from 0 to n - m:
   if Hp == Ht:
```

```
        match_found = true
        for j from 0 to m - 1:
            if T[i + j] != P[j]:
                match_found = false
                break

        if match_found == true:
            return i  // Pattern found at index i

    // Roll the hash for the next window
    if i < n - m:
        Ht = (d * (Ht - T[i] * h) + T[i + m]) % q

        if Ht < 0:
            Ht = Ht + q

return -1 // Pattern not found
// Pseudocode end
```

Summary of Programming Challenge Objectives

The general idea of the programming challenge is that given the long DNA genome string, recognize certain patterns/mutations in the genome. The usage of the Rabin-Karp algorithm in this case is relatively clear, as the easy solution would just be to run the algorithm over the genome for each mutation pattern. Assuming the genome is length n and there are k mutations with average length m, this would give an average time complexity of $O((n + m)k)$.

However it would be unfair to the Rabin-Karp algorithm just to show this use case. When searching for a single pattern in a string, most would opt for the KMP algorithm because it avoids the slow worst case example. However Rabin-Karp is useful for multiple pattern search:

given a string and a set of k equal length patterns, find the index of each occurrence of a pattern in the string. Instead of running the algorithm for each pattern, we can hash each pattern beforehand, put the hash into a set, and then instead of checking at each iteration if substring hash is equal to a pattern hash, we check if the substring hash is in the set. If it is, we find which string(s) it correlates to, and compare it to that string. This gives us an average time complexity of $O(n + km)$ which is usually much faster than the previous approach. This works especially well when the length of the string is much greater than the length of the patterns, which is true in our large test cases for this assignment.

However there is one key detail I am leaving out: the mutations are not all the same length! So does this mean this algorithm is useless? Well no. Most mutations in our test cases are of similar length, so the algorithm is still useful. For example a test case with 1000 mutations would take a lot of time to check, but if we find out that every mutation is of length 6, 7, 8, or 9 then we could run our multi-string algorithm only 4 times. So I'll introduce a new variable, c, which denotes the number of unique lengths of our mutations, and we get an average time complexity of $O(cn + km)$.

So in conclusion the main learning objective of this assignment is to get the student familiar with multi-pattern search for the Rabin-Karp algorithm, as in the typical single-pattern case there's no real reason to use Rabin-Karp.

One last point of interest is that by default most implementations of the Rabin-Karp algorithm use 256 as their base for the hash function, as each char is 1 byte = 8 bits = 256 possibilities, but since there are only 4 possibilities for DNA (A,T,G,C) we can use 4 as our base.

Key Ideas for Solving Programming Challenge

I think after introducing the idea of multi-pattern search in the slides, it will be relatively clear that the intention of this assignment is to test that. I think the biggest hurdle will be the student trying to figure out how to implement it in a way that satisfies multiple different pattern lengths. This is challenging because it requires a lot of initial thinking, but I think if a student sees the details in the input (up to 1000 strings but only 32 possible lengths) then they would be able to figure out that they could run the algorithm on each length of string. At this point the solution is quite simple. That's why this change is interesting and challenging, because the student will have to solve this problem while maintaining a low time complexity, but the solution itself is not too difficult to implement once they overcome this hurdle.

The time complexity of the optimal solution (with multi-pattern matching) would be $O(cn + km)$ as described above, where k is the amount of mutations, m is their average length, n is the length of the genome, and c is the amount of unique lengths of the mutations.

Conclusion

The Rabin-Karp algorithm provides a highly optimized solution to the string matching problem by leveraging mathematical hashing to bypass the inefficiencies of naive, brute-force approaches. By converting characters into integer hash values and dynamically updating those values using a rolling hash, the algorithm reduces the cost of substring comparisons to constant time. This yields a highly efficient average overall time complexity of $O(n + m)$.

While the standard implementation is excellent for single-pattern detection, its true power and utility are unlocked when adapted for multi-pattern search scenarios, such as the genome mutation scanning detailed in our custom programming challenge. By grouping target patterns by length, hashing them into a set, and adjusting the alphabet base size, the algorithm scales to handle massive datasets and numerous targets simultaneously.

Ultimately, Rabin-Karp remains a foundational algorithm in computer science, demonstrating how simple mathematical substitutions and dynamic programming principles can achieve large optimizations in computational performance.

References

"Naive Algorithm for Pattern Searching." GeeksforGeeks, https://www.geeksforgeeks.org/dsa/naive-algorithm-for-pattern-searching/.

"Rabin-Karp Algorithm for Pattern Searching." GeeksforGeeks, https://www.geeksforgeeks.org/dsa/rabin-karp-algorithm-for-pattern-searching/.

"What Is the Rabin-Karp Algorithm?" YouTube, uploaded by Abdul Bari, https://www.youtube.com/watch?v=yFHV7weZ_as.

"Rabin–Karp Algorithm." Wikipedia, https://en.wikipedia.org/wiki/Rabin%E2%80%93Karp_algorithm.